

SMART TABLEKEEPER

AI-Powered Restaurant Reservation, Discovery & Intelligence Platform

Product Requirements Document (PRD) + Academic PG/PhD Project Proposal

23 September 2026



Purpose: a consolidated build specification, academic proposal, implementation roadmap, business-logic reference, software-factory guide, and presentation document.

Document Index

1. Document Purpose
2. 1. Executive Summary
3. 2. Problem Statement
4. 3. Product Vision
5. 4. Target Users & Personas
6. 5. Core User Journey
7. 6. Reservation Input Parameters
8. 7. Budget Estimation & Recommendation Logic
9. 8. Restaurant Result Card
10. 9. Functional Requirements
11. 10. Non-Functional Requirements
12. 11. Smart Features / Product Differentiators
13. 12. Business Logic
14. 13. Recommendation Engine
15. 14. Restaurant Analytics / Business Intelligence
16. 15. Scheduled Booking Agent
17. 16. Transportation Integration
18. 17. Hackathon Stage Roadmap
19. 18. BAND Software Factory
20. 19. Agent Responsibilities
21. 20. Proposed Technical Stack
22. 21. Proposed Data Model
23. 22. API Strategy
24. 23. Testing Strategy
25. 24. Docker & Deployment
26. 25. Repository Structure
27. 26. Implementation Priority
28. 27. Academic PG/PhD Project Framing
29. 28. Security, Privacy & Responsible Data Use
30. 29. Risks & Mitigations
31. 30. Definition of Done
32. 31. Demo & Presentation Storyline
33. 32. Copy-Paste Factory Mandates
34. 33. Factory Task Template
35. 34. Pre-Kickoff / Kickoff / Submission Timeline
36. 35. Final Product Scope Summary

Document Purpose

This document consolidates the project brainstorming, product requirements, business logic, implementation plan, software-factory workflow, technical architecture, testing strategy, deployment approach, and presentation plan developed for the Smart Tablekeeper project.

It is designed to serve two purposes: (1) a build-ready PRD for the hackathon implementation and (2) an academic project/proposal document suitable for PG/PhD-style project presentation, review, or submission.

Where the hackathon's official Stage 1 specification is not yet available in the project context, exact endpoint names, response schemas, error codes, and Stage 4 requirements are intentionally left as specification-driven placeholders rather than invented.

1. Executive Summary

Smart Tablekeeper is an AI-powered restaurant reservation and discovery platform that converts a user's natural-language dining intent into structured reservation requirements, checks availability, recommends suitable restaurants and time slots, handles unavailable requests through intelligent alternatives and waitlists, protects bookings against concurrency and retry failures, and provides restaurant-side insights from demand and feedback.

The central product idea is to move beyond a basic restaurant booking form. The system understands what the diner wants, matches availability and budget, presents transparent restaurant information such as rating and estimated spend, and provides recovery paths when the preferred option is unavailable.

The product has three core reliability and experience pillars: Understand, Recover, and Guarantee. A fourth business pillar, Improve, uses aggregated demand and feedback signals to help restaurants understand customer needs.

2. Problem Statement

Traditional reservation flows require users to manually enter multiple fields and repeatedly search when a preferred slot is unavailable. A failed search often ends with a simple 'no availability' result, creating friction and dead ends.

Restaurants also receive valuable demand signals—what users search for, preferred times, party sizes, budgets, occasions, seating preferences, and feedback—but these signals are rarely converted into actionable operational insights.

Smart Tablekeeper addresses both sides: it helps diners find and secure an appropriate dining option while helping restaurants understand demand and improvement opportunities.

3. Product Vision

Provide an intelligent dining assistant that can understand a user's intent, match it against real reservation constraints, explain recommendations, recover from unavailable slots, and create reliable bookings without duplicate reservations.

Long-term vision: become a decision-support and reservation layer connecting diners, restaurants, availability, feedback, and optional post-booking services such as transportation.

4. Target Users & Personas

Diner: wants a table for a group, date/time, occasion, seating preference, location, and budget. The diner values convenience, transparency, availability, and relevant recommendations.

Restaurant operator: wants reservations to be reliable and wants aggregated insight into demand, booking conversion, customer preferences, and feedback.

Platform administrator/operations user: monitors system health, reservation integrity, agent activity, and analytics.

Software-factory agents: Planner, Builder, and Verifier collaborate inside BAND Desktop to transform the official specification and product requirements into tested software.

5. Core User Journey

- 1. User enters a natural-language request or structured reservation parameters.
- 2. AI Booking Assistant extracts party size, date, preferred time, occasion, seating preference, location, and budget.
- 3. System validates required information and asks for missing critical fields.
- 4. Reservation/Recommendation Engine checks restaurant availability, table capacity, budget compatibility, preferences, location, and relevant quality signals.
- 5. Matching restaurants and time slots are displayed with availability, rating/review count, estimated spend, distance, seating information, and booking action.
- 6. User selects an option and confirms the booking.
- 7. Concurrency and idempotency controls ensure that a retry or simultaneous request cannot create duplicate confirmed reservations.
- 8. If the preferred option is unavailable, the system offers alternative slots/restaurants or a waitlist path.
- 9. After booking, the user can view reservation history and optional directions/transport options.
- 10. Aggregated searches, bookings, and feedback feed the restaurant analytics layer.

6. Reservation Input Parameters

Party size — number of diners.

Date — requested dining date.

Preferred time — exact or approximate time.

Occasion — examples include birthday, anniversary, family, date, or business.

Seating preference — examples include quiet, outdoor, window, family-friendly, or other supported preferences.

Location — preferred area or distance constraint.

Budget — maximum or target planned dining spend for the group.

Restaurant preference — optional named restaurant or restaurant attributes.

Natural-language request — free-form expression that can contain any combination of the above.

7. Budget Estimation & Recommendation Logic

Budget is a first-class constraint in the recommendation engine. The system should not invent an exact bill.

A restaurant can provide an average spend per person, or the platform can calculate a historical average from appropriate transaction/order data. Example: average spend **■**520 per person for a party of 5 gives an estimated group spend of **■**2,600.

The interface must label this as an estimate, not a guaranteed bill, because actual spending depends on what the group orders and applicable taxes, service charges, or other restaurant-specific factors.

The recommendation engine can compare estimated group spend with the user's budget. A restaurant may be considered budget-compatible when its estimated spend is at or below the user's budget, or it may be shown as a near-budget option with a clear explanation if product policy permits.

Budget should not be used to favor restaurants because of commercial relationships. Recommendations should be driven by user relevance, availability, budget fit, preference match, distance, rating/review signals, and other documented product rules.

8. Restaurant Result Card

Each recommendation should expose enough information for an informed choice without forcing the user to open multiple screens.

Recommended display: restaurant name, rating, review count, cuisine/type, estimated spend for the user's party, average spend per person, available time slot, seating match, distance, occasion suitability tags where supported, and a Book Table action.

Example: Restaurant A — 4.6/5 (1,248 reviews) — Indian/Family Dining — ~\$2,600 for 5 people — 7:00 PM available — Quiet seating — 2.4 km — Book Table.

Rating is a quality signal shown in the results; it is not a required user input.

9. Functional Requirements

- FR-01: Accept natural-language reservation requests.
- FR-02: Convert natural-language input into structured reservation requirements.
- FR-03: Validate required reservation fields and handle missing/invalid values.
- FR-04: Search availability based on date/time and party size.
- FR-05: Consider budget compatibility in recommendations.
- FR-06: Consider seating, occasion, location, and restaurant preferences where data is available.
- FR-07: Display restaurant ratings/reviews as result information.
- FR-08: Recommend alternative time slots when the requested slot is unavailable.
- FR-09: Recommend alternative restaurants when the requested restaurant is unavailable.
- FR-10: Support waitlist registration and availability notifications.
- FR-11: Require user confirmation before a waitlist offer becomes a booking in the initial implementation.
- FR-12: Prevent double booking under concurrent requests.
- FR-13: Make booking operations idempotent so safe retries return the original result.
- FR-14: Return documented validation and business errors instead of generic 500 responses for malformed input.
- FR-15: Maintain reservation history.
- FR-16: Support scheduled booking as an advanced feature after the core stages are stable.
- FR-17: Collect appropriate demand and feedback signals for restaurant analytics.
- FR-18: Provide restaurant-side insight summaries such as demand patterns, booking requests, conversion, preference trends, and feedback themes.
- FR-19: Support post-booking directions/transport options as an optional future integration.
- FR-20: Preserve all earlier-stage functionality when extending the domain in Stage 4.

10. Non-Functional Requirements

Reliability: reservation state must remain correct under retries and concurrent requests.

Consistency: confirmed reservation state must not permit overlapping confirmed bookings for the same table/resource.

Idempotency: repeated requests with the same idempotency key must not create duplicate reservations.

Validation: malformed input must produce documented validation/business errors.

Performance: availability and recommendation operations should respond quickly enough for interactive use; exact targets should be set after the official specification is available.

Security: validate and sanitize inputs, protect credentials, enforce authorization boundaries, and avoid exposing unnecessary personal data.

Observability: log important reservation attempts, failures, idempotency events, and agent outcomes without logging secrets.

Offline build: the hackathon solution must be able to build and run in a clean Docker environment without outbound network access during the required build/run process.

Maintainability: implementation should be modular, tested, documented, and structured so later stages extend earlier work without breaking it.

11. Smart Features / Product Differentiators

AI Booking Assistant — natural-language intent extraction.

Smart Slot Recommendation — suggest useful available times rather than requiring repeated manual searches.

Smart Fallback — when the requested slot is unavailable, find nearby times or alternative restaurants matching the request.

Smart Waitlist — monitor unavailable preferred slots and notify the user when a matching opportunity becomes available; initial flow is Notify → User Confirms → Book.

Budget-Aware Discovery — incorporate user budget and estimated restaurant spend.

Rating-Aware Results — show rating and review count below/alongside the reservation details in recommendation cards.

Group-Aware Booking — account for party size and suitable table capacity.

Occasion-Aware Booking — adapt recommendations to birthday, anniversary, family, business, or other supported occasions.

Scheduled Booking Agent — user sets a future attempt time; the agent wakes, checks availability, attempts booking, and follows fallback/notification logic if unavailable.

Restaurant Recommendation Engine — combines availability, budget, preferences, distance, and quality signals.

Restaurant Business Intelligence — aggregate demand and feedback into actionable restaurant reports.

Transportation Integration — optional post-booking directions/ride assistance.

12. Business Logic

Availability rule: a table/resource is eligible only when it can accommodate the requested party and does not conflict with another confirmed reservation or active booking hold according to the official specification.

Overlap rule: reservation intervals for the same resource must not overlap when the domain's reservation model defines exclusive occupancy.

Idempotency rule: the same logical booking request with the same idempotency key must return the original result rather than creating a second reservation.

Concurrency rule: two simultaneous valid requests competing for the same resource cannot both be confirmed.

Fallback rule: when the requested option is unavailable, search alternatives using the user's stated constraints and transparent ranking signals.

Waitlist rule: a user can register interest in an unavailable slot. When a matching opening appears, notify the user; the initial design requires explicit user confirmation before booking.

Scheduled-agent rule: at the configured time, attempt the requested booking. If unavailable, execute configured fallback/waitlist/notification behavior and record the outcome.

Recommendation rule: never recommend a slot as available unless the availability engine confirms it.

Budget rule: estimated group spend = restaurant average spend per person × party size, subject to the restaurant/platform's data policy.

Analytics rule: restaurant reports should use appropriate aggregated/authorized data and should not expose unnecessary private user information.

13. Recommendation Engine

Candidate generation: identify restaurants and slots satisfying hard constraints such as date, party size, and availability.

Filtering: apply budget, location, seating, occasion, and restaurant preferences as applicable.

Ranking: rank candidates using transparent relevance signals such as availability fit, budget fit, preference match, distance, rating/review signals, and time proximity.

Explanation: show users why an option matches, for example 'Within budget', 'Quiet seating available', '7:00 PM available', or '2.4 km away'.

Commercial neutrality principle: commercial participation should not override user fit or availability.

14. Restaurant Analytics / Business Intelligence

Collect appropriate aggregates from searches, booking requests, completed bookings, cancellations, preferences, and feedback.

Monthly or periodic restaurant report can include: demand volume, requested times, party-size patterns, budget bands, booking conversion, top requested preferences, positive feedback themes, and potential improvement areas.

The system can compare high-demand attributes with lower-performing restaurant attributes to generate actionable suggestions, while clearly presenting such output as analytics rather than guaranteed causal conclusions.

Example insight: repeated searches for quiet seating during weekend dinner hours may indicate an opportunity to improve or expose quiet-seating inventory.

The analytics module is intended to help restaurants improve service and discoverability, not merely to create paid ranking.

15. Scheduled Booking Agent

User creates a scheduled booking request in advance with restaurant/restaurant criteria, party size, target date/time, budget, and relevant preferences.

At the scheduled attempt time, the agent checks current availability.

If available, it attempts a normal idempotent booking operation.

If unavailable, it follows configured fallback logic such as alternative slots, alternative restaurants, waitlist, or user notification.

Every attempt should record timestamp, request identifier, result, and failure/retry information.

This feature is an advanced enhancement and should only be implemented after the core hackathon stages are stable and passing.

16. Transportation Integration

After a confirmed booking, the application can offer directions or an optional ride arrangement.

A provider integration can be added later, subject to API access, terms, and hackathon rules.

Transportation failure must not invalidate an otherwise successful restaurant reservation.

17. Hackathon Stage Roadmap

Stage 1 — JSON API: implement every endpoint, response shape, and documented error in the official specification. This is the minimum eligibility stage. Exact routes must come from the official spec.

Stage 2 — Web UI: build the UI against the Stage 1 API and implement every required data-testid exactly as specified.

Stage 3 — Concurrency: prevent double booking, make retries idempotent, and ensure malformed input returns documented errors rather than 500 responses.

Stage 4 — Domain Extension: extend the same domain and existing model/API, preserve prior functionality, and satisfy the official Stage 4 extension specification.

Important: do not invent Stage 1 endpoints or Stage 4 requirements before the official specification is released.

18. BAND Software Factory

The project is built through a BAND Desktop software factory rather than treating the application as a one-off manual coding project.

Use at least three distinct coding-agent seats: Planner/Architect, Builder/Developer, and Verifier/QA.

Each seat receives a generic permanent mandate. Track-specific requirements belong in tasks supplied to the agents, not in permanent mandate text.

Factory loop: REQUIREMENT → PLANNER → TASKS → BUILDER → CODE → VERIFIER → PASS/FAIL → if FAIL, BUILDER → VERIFIER → if PASS, DONE.

Every task should produce evidence: task identifier, changed files, tests run, results, API validation, and commit or equivalent repository evidence.

19. Agent Responsibilities

Planner: inspect requirements/repository, identify constraints and dependencies, break work into small ordered testable tasks, define acceptance criteria and test expectations, and flag ambiguity instead of inventing behavior.

Builder: inspect existing code before changes, implement small testable changes, preserve passing functionality, run validation, report changed files and evidence, and escalate missing requirements.

Verifier: independently inspect implementation, run tests, check requirements, edge cases, validation, error handling, concurrency/retry behavior, and build/runtime readiness; return a structured PASS/FAIL report with evidence.

20. Proposed Technical Stack

Frontend: React with a modern component/UI system. Vue is an alternative, but the implementation should select one framework and remain consistent.

Backend: Python FastAPI is recommended for a clean API and AI/service integration. Node.js/Express is a viable alternative if chosen consistently.

Database: PostgreSQL for reservation, restaurant, table, user, waitlist, scheduling, and analytics data.

Caching/realtime: Redis is optional for caching, distributed locks, queues, or ephemeral state; WebSockets can be used for live availability/notification updates.

AI/NLP: an LLM-backed extraction/recommendation service with structured output validation.

Containerization: Docker / Docker Compose as appropriate.

Version control: GitHub.

Architecture principle: keep the Stage 1 API contract isolated from optional product enhancements so the official harness remains stable.

21. Proposed Data Model

User: identity and user preferences needed for reservations.

Restaurant: restaurant identity, location, cuisine/type, rating/review summary, average spend per person, supported features.

Table/Resource: capacity, seating attributes, restaurant relationship, and availability metadata.

Reservation: user, restaurant, table/resource, party size, date/time, status, idempotency key/reference, timestamps.

WaitlistEntry: user, restaurant/resource criteria, requested date/time, party size, preferences, status, notification state.

ScheduledBooking: user, target request, scheduled attempt time, status, retry/attempt history.

Feedback/Review: restaurant association, rating/feedback content or authorized aggregate, timestamps.

RestaurantInsight: aggregated metric/report output and reporting period.

Exact fields must be reconciled with the official Stage 1 API/data model before implementation.

22. API Strategy

The API should be contract-first against the official hackathon specification.

Do not invent endpoint names, request/response shapes, status codes, or error schemas before the official Stage 1 specification is available.

After the official spec is supplied, the Planner should convert each endpoint and documented behavior into explicit implementation and verification tasks.

Product extensions such as recommendations, waitlist, scheduled booking, and analytics should be layered without breaking the required Stage 1 contract.

23. Testing Strategy

Unit tests: validation, parsing, budget calculation, recommendation filtering/ranking helpers, and domain rules.

API tests: every required endpoint, success cases, validation errors, documented business errors, and response schema.

Integration tests: database + reservation service + API flows.

Concurrency tests: simultaneous attempts against the same resource; assert at most one conflicting booking is confirmed.

Idempotency tests: send the same booking request repeatedly with the same idempotency key; assert one logical reservation and consistent original response.

Malformed-input tests: invalid dates, missing required fields, invalid party sizes, malformed payloads, and unsupported values should produce documented errors.

Regression tests: every later stage must preserve earlier-stage behavior.

Clean-build test: build the Docker image from a clean environment and run the service without relying on undeclared local state.

24. Docker & Deployment

Provide a reproducible Docker build and startup path.

Validate the application from a clean container environment.

The hackathon context requires Dockerized service behavior and no outbound network dependency during the relevant clean build/run process.

For a public demo, deploy the Dockerized service to a hosting environment allowed by the hackathon and expose a stable demo URL. Exact hosting choice should be confirmed against the current hackathon rules.

25. Repository Structure

Recommended structure:

wearedevelopers-ai-dark-factory/

■■■■ stage-1/ (application, tests, Docker/support files)

■■■■ stage-2/ (application, tests, Docker/support files)

■■■■ stage-3/ (application, tests, Docker/support files)

■■■■ stage-4/ (application, tests, Docker/support files)

■■■■ mandates/ planner.md, builder.md, verifier.md

■■■■ factory-description.md

■■■■ README.md

■■■■ docs/ PRD, architecture, API notes, test evidence

26. Implementation Priority

Tier 0 — Factory foundation: BAND room, three seats, generic mandates, handoff protocol, evidence format, GitHub repository, Docker baseline.

Tier 1 — Hackathon core: official Stage 1 API → Stage 2 UI → Stage 3 concurrency/idempotency → Stage 4 domain extension.

Tier 2 — Product differentiation: natural-language assistant, budget-aware recommendations, smart alternatives, waitlist, rating-aware result cards, restaurant analytics.

Tier 3 — Advanced enhancements: scheduled booking agent, richer real-time updates, transportation integration.

Do not allow Tier 2/3 features to destabilize or delay the required hackathon stages.

27. Academic PG/PhD Project Framing

Proposed academic title: 'Smart Tablekeeper: An AI-Driven Constraint-Aware Restaurant Reservation and Recommendation System with Reliable Concurrent Booking'.

Research/problem focus: intelligent reservation systems must combine natural-language intent understanding with hard availability constraints, budget-aware recommendation, reliable concurrent booking, and feedback-driven optimization.

Objectives: develop an intent extraction layer; design a constraint-aware recommendation engine; implement reliable reservation concurrency/idempotency; create fallback and waitlist mechanisms; build restaurant analytics; evaluate usability, recommendation relevance, booking reliability, and system performance.

Research contribution framing: the project integrates conversational intent, constraint-based reservation matching, reliability mechanisms, and restaurant-side analytics into one workflow. The strongest technical evaluation areas are booking correctness under concurrency, retry safety, recommendation relevance, and user task completion.

Evaluation metrics can include: booking success rate, conflicting-booking rate, idempotency correctness, response latency, recommendation acceptance/click-through, budget-fit rate, fallback success rate, waitlist conversion, and user satisfaction. Exact experimental methodology should be finalized with the academic supervisor.

Academic limitation: the project should not claim causal effects or real-world generalization without appropriate data, experiments, and evaluation.

28. Security, Privacy & Responsible Data Use

Collect only information required for the reservation and analytics purpose.

Protect authentication credentials, API keys, and provider tokens through environment/configuration management rather than source code.

Avoid exposing private user data in restaurant reports.

Use aggregated or appropriately authorized feedback for business intelligence.

Validate all user-provided fields and treat AI-generated structured data as untrusted until schema validation succeeds.

29. Risks & Mitigations

Risk: official Stage 1 API differs from assumptions. Mitigation: treat the official specification as the source of truth and regenerate tasks after kickoff.

Risk: overbuilding before mandatory stages pass. Mitigation: enforce Tier 0/Tier 1 priority.

Risk: double booking. Mitigation: transactional reservation logic, appropriate locking/constraints, and concurrency tests.

Risk: duplicate bookings on retries. Mitigation: idempotency keys and persisted idempotency result handling.

Risk: inaccurate budget estimate. Mitigation: label estimates clearly and base them on restaurant-provided or historical average-spend data.

Risk: recommendation bias toward commercial partners. Mitigation: user-fit and availability rules take precedence.

Risk: AI extracts incorrect fields. Mitigation: structured schema validation, confidence/validation checks, and user confirmation for critical information.

Risk: optional integrations consume schedule. Mitigation: implement advanced integrations only after core stages pass.

30. Definition of Done

A stage is not complete because code exists; it is complete when requirements are implemented, tests pass, Docker build/run succeeds, regression behavior is preserved, and the Verifier provides evidence-based approval.

For each task: requirement reference, implementation, tests, test output, changed files, known limitations, and verifier result.

For final submission: all required stages are present and buildable, the service is Dockerized, harness checks pass, the BAND room/workflow is exported as required, GitHub is public, and the demo/presentation artifacts are complete.

31. Demo & Presentation Storyline

- 1. Introduce the problem: reservation search is manual and unavailable slots create dead ends.
- 2. Show the BAND Desktop software factory and the three generic coding-agent roles.
- 3. Show how the official requirement/specification becomes Planner tasks, Builder implementation, and Verifier evidence.
- 4. Demonstrate a natural-language request containing party size, date/time, occasion, seating, location, and budget.
- 5. Show recommendation cards containing availability, rating/review count, estimated spend, and distance.
- 6. Demonstrate unavailable-slot fallback and waitlist.
- 7. Demonstrate concurrent booking attempts where only one valid conflicting reservation succeeds.
- 8. Demonstrate idempotent retry behavior.
- 9. Show restaurant analytics/insights.
- 10. Show Dockerized deployment and GitHub stage structure.
- 11. Close with the academic/technical contribution: intent understanding + constraint matching + reliable booking + feedback-driven improvement.

32. Copy-Paste Factory Mandates

Planner mandate: 'You are a software architecture and planning agent. Analyze supplied requirements and the repository; identify constraints, dependencies, risks, and acceptance criteria; break work into small, ordered, testable tasks; define implementation and verification expectations; avoid inventing unspecified behavior; flag ambiguities; produce structured tasks with inputs, outputs, dependencies, acceptance criteria, and evidence requirements.'

Builder mandate: 'You are a software implementation agent. Take clearly defined implementation tasks. Inspect existing code before modifying it. Implement changes in small, testable increments. Preserve existing functionality unless the task explicitly requires a change. Follow repository conventions and supplied requirements. Run appropriate tests and validation. Do not claim completion without evidence. Report files changed, tests run, failures, and remaining risks. Escalate when required information is missing.'

Verifier mandate: 'You are a software verification and quality agent. Inspect the implementation against supplied requirements and acceptance criteria. Run relevant unit, integration, API, regression, concurrency, and build/runtime tests as applicable. Never approve based only on the implementer claim. Produce a structured PASS/FAIL report with evidence, exact failures, reproduction details, and required fixes. Flag ambiguous requirements rather than inventing behavior.'

These mandates are intentionally generic and do not hardcode Tablekeeper endpoints or stage-specific implementation details.

33. Factory Task Template

PROJECT SPECIFICATION: <paste official specification>

CURRENT REPOSITORY: <repository/branch/context>

OBJECTIVE: <specific stage/task>

CONSTRAINTS: follow official specification; do not invent unspecified behavior; preserve passing functionality; satisfy Docker/offline requirements; provide evidence.

PLANNER OUTPUT: requirements, dependencies, ordered tasks, acceptance criteria, test plan, risks.

BUILDER OUTPUT: implementation summary, files changed, tests run, results, remaining risks.

VERIFIER OUTPUT: PASS/FAIL, requirement-by-requirement evidence, failed cases, reproduction steps, required fixes.

34. Pre-Kickoff / Kickoff / Submission Timeline

Before official kickoff: prepare BAND room, three seats, generic mandates, GitHub repository, Docker baseline, documentation templates, and factory task templates.

At kickoff: ingest the official Stage specification into the Planner; do not assume undocumented endpoints or schemas.

Next: complete Stage 1 through Planner → Builder → Verifier loops.

Then: Stage 2 UI and required data-testid validation.

Then: Stage 3 concurrency/idempotency and malformed-input verification.

Then: Stage 4 domain extension while preserving all earlier functionality.

Finalization: clean Docker build/run, full test suite, harness checks, repository cleanup, BAND room export, demo URL, cover image, demo video, slides, project description, and final submission.

35. Final Product Scope Summary

Core reservation system: availability, booking, cancellation/history as required by the official API.

AI experience: natural-language reservation intent extraction.

Decision support: budget-aware, rating-aware, preference-aware restaurant recommendations.

Recovery: alternative slots/restaurants and waitlist.

Reliability: concurrency protection and idempotency.

Automation: scheduled booking agent after core stability.

Business intelligence: restaurant demand and feedback insights.

Future integration: transportation assistance.

Engineering foundation: BAND software factory, generic agent mandates, evidence-based verification, Docker, GitHub, and staged delivery.